Interface, Instruction Adoption, and Forgetting in Small Computer-Use Agents: A Diagnostic Case Study

Wenbo Yan · EURECOM, Sophia Antipolis, France · yan2000926@gmail.com

October 2026 · v0.7

Abstract

Small computer-use agents fail for reasons that are easy to misattribute. We report a diagnostic case study of a 4B agent on file editing, browser fixtures, and native macOS tasks, probing each failure class with a separate intervention. **Interface.** Making unchecked checkbox state explicit raised browser-settings success from 1/18 to 14/18 executions (all six tasks improved; exact task-level sign-flip $p = 1/32$), and replacing whole-text editing with line-oriented tools raised TextEdit success from 4/18 to 17/18. The tool change also held for a second 4B student (0/18 to 16/18) and, in a pre-registered confirmation on six new TextEdit tasks, for both students (3/18 to 18/18 and 2/18 to 17/18; $p = 1/32$ each). **Presentation.** A five-task pilot suggested that the same teacher rule is ignored in the system prompt but used beside the task with an application plan. A pre-registered confirmation on ten new Calculator tasks separated the two factors: the rule in the system prompt gave 6/30 executions, beside the task without a plan 19/30 (8 of 10 tasks improved, $p = 1/128$), and beside the task with a plan 23/30 (versus no rule 1/30, $p = 1/512$); the plan's increment over placement alone was not detectable ($p = 7/32$). Placement also brought over-application: on two tasks where the rule does not apply, task-adjacent conditions scored 8/12 against 18/18 without them. The presentation effect did not appear with the second student, which could not operate Calculator in any condition. **Consolidation.** A rank-16 LoRA adapter trained on 1,677 action-level examples from guided rollouts raised file success on new instances of the trained families from 30/72 to 64/72 (14 of 24 tasks improved, 2 declined) while success on two untrained families fell from 15/36 to 1/36 (6 of 12 declined, 1 improved). Follow-up variants rule out restoring reasoning text as a sufficient fix but do not identify a single mechanism. A composite desktop configuration reached 41/45; only its TextEdit component has a matched control, which reached the same score without lessons. Apart from the two pre-registered desktop confirmations, all results are development-set results with one training seed and nominal, uncorrected task-level tests. We offer a diagnostic order—interface, then presentation of guidance, then retention—as a working procedure, not as a tested algorithm.

1. Introduction

A personal agent built around a small local model should benefit from experience without becoming less reliable on unfamiliar requests. Failures, however, admit several explanations. The model may lack a procedure, fail to use a retrieved procedure, misinterpret the observation, or be unable to express a correct action through the available tool. Each explanation suggests a different repair: collect demonstrations, change how guidance is presented, change the observation or action space, or change the weights. Choosing the wrong layer wastes data, teacher calls, and training cycles, and it can produce a confident but incorrect claim that the model "cannot" do a task.

We study this attribution problem in Kage, a single-user agent whose student edits files, operates local browser fixtures, and interacts with macOS applications through the Accessibility API. The experiments combine automated teacher calls, frozen benchmarks with programmatic checkers, developer-designed tools and prompts, and a consolidation stage that turns verified guided rollouts into LoRA training data. The study is deliberately narrow: a primary 4B student, a partial replication with a second 4B student on the desktop suite, one training seed, procedural task generators, and development-set evaluations.

Our claims, each with its evidence and its limit, are:

- **C1 (interface).** Failures that persisted through demonstrations, rules, and consolidation disappeared when the observation or action representation changed. Explicit checkbox state moved browser settings from 1/18 to 14/18 executions (six of six tasks improved). Line-oriented editing tools moved TextEdit from 4/18 to

17/18 for the primary student and from 0/18 to 16/18 for the second student (six of six tasks improved in each), and the effect held on six new, pre-registered tasks for both students.

- **C2 (adoption).** Retrieving guidance is not the same as using it. Relevant demonstrations and rules were present yet did not change browser behavior. On the desktop, a pilot and then a pre-registered five-condition confirmation on new Calculator tasks showed that the primary student largely ignored a rule in the system prompt (6/30) and used it when the rule sat beside the task (19/30), with or without an application plan (23/30); the plan added little that we could detect. Placement also made the student apply the rule where it did not hold. The effect did not reproduce with the second student, whose Calculator failures no presentation condition repaired.
- **C3 (forgetting).** Supervised fine-tuning on the student's own verified successes improved trained families and degraded untrained ones in the same adapter. Variants that filtered targets, added replay, or restored teacher-written rationales did not recover both, and they do not identify a single cause.
- **Working procedure.** Rule out observation and action-interface causes, then the presentation of guidance, then retention, before attributing a failure to capability (Figure 1). This order organized our investigation; we did not compare it with other orders.

We do **not** claim autonomous recursive self-improvement, a general property of 4B models, or a causal decomposition of the final desktop configuration. The interface changes were designed by the developer, not by the agent. Section 10 states the boundaries in detail.

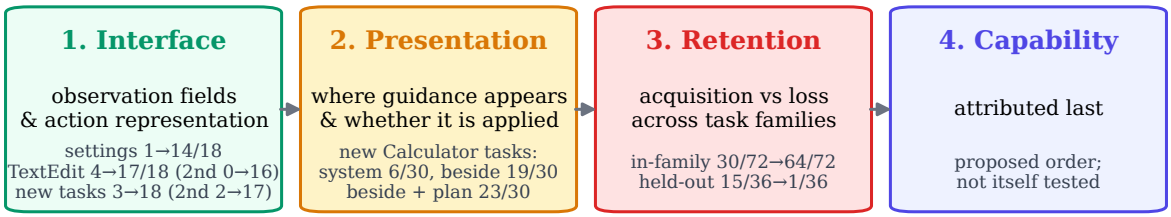


Figure 1: Working procedure used in this study. Each box lists the layer examined and its principal evidence; parenthesized values are from the second student. The order is a proposed procedure, not an experimentally compared one.

2. System and evidence boundaries

Figure 2 shows the components and the information each one receives. The student acts only through the interface; the checker evaluates the environment outside the student's prompt; the teacher sees failures and produces guidance or guided rollouts; consolidation turns verified rollouts into an adapter that is evaluated without guidance.

Students. The primary student is Agents-A1-4B in Q4_K_M format (frozen revision `d92b02e2`, GGUF SHA-256 `d93c393a…f062`). File and browser evaluations ran on Kaggle T4 GPUs with a prebuilt llama.cpp (context 8,192 tokens for files, 16,384 for the browser); desktop evaluations ran on an Apple M4 Mac with llama.cpp 0.4.1 (build 10964, context 8,192). The second student, used only for the desktop replication in Sections 4–5, is NVIDIA Nemotron-3-Nano-4B in Q4_K_M format (GGUF SHA-256 `be5d9a65…5ef2`) under the same server flags. Consolidation training ran on a Colab A100.

Execution budgets. The file benchmark permits five loop steps, six local model requests, twelve tool dispatches, and 150 s per attempt. The browser protocol uses eight steps and ten model requests. Desktop tasks have task-specific limits of six to twenty steps in the Desktop-v1 manifest, and the desktop loop issues one model request per step. These are different protocols, not a common compute budget. File and desktop runs use temperature 0.7; no result depends on temperature-zero determinism.

Learning and guidance. After a failed file or browser attempt, a teacher demonstration can be stored as trajectory memory or converted into a parameterized skill verified in a sandbox. A TF-IDF gate selects items by instruction similarity; when nothing passes, the request is byte-identical to the raw student's. A separate mechanism asks the

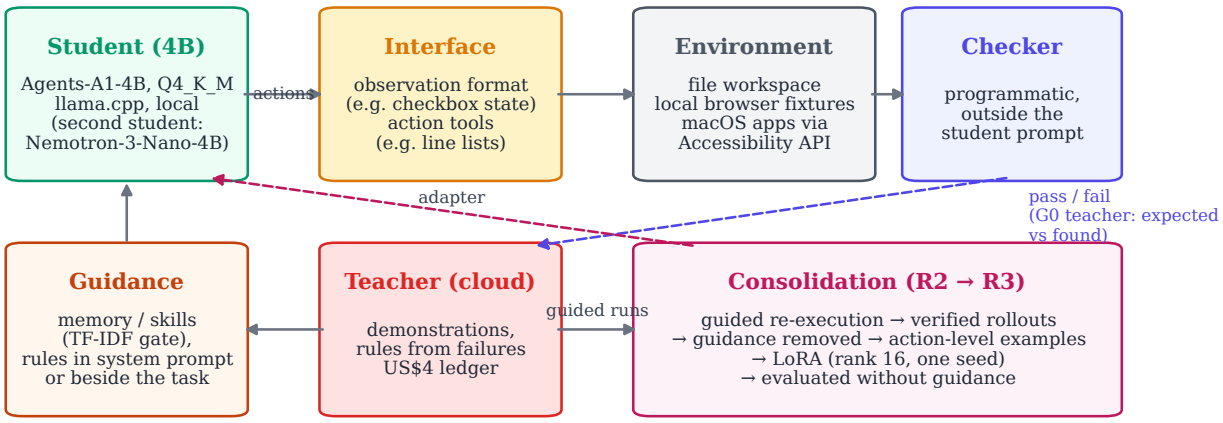


Figure 2: System overview. Solid arrows are the per-task loop; dashed arrows are training-time paths (checker verdicts to the teacher, and the trained adapter back to the student).

teacher for one or two general rules, which are placed in the system prompt or beside the task depending on the condition.

Checker and teacher boundaries. Programmatic checkers evaluate outcomes outside the student's prompt. In the file and browser experiments the teacher sees the checker's pass/fail verdict, not hidden expected values. The desktop teacher is different: its prompts include the training task's expected result and the observed checker finding, so it receives privileged training feedback, not merely a binary reward; the student receives the general rules, not test answers. The four desktop rules were written from failures on four of the ten training tasks (three TextEdit tasks and one Calculator chain task). Student-unseen tasks may be developer-seen: inspecting evaluation failures informed interface and prompt revisions, and the five desktop variants were reused throughout development.

Consolidation. R2 collected guided rollouts on file and browser tasks. Successful rollouts were converted to action-level supervised examples after removing the guidance, split by task into training and validation sets, and used for LoRA training (rank 16 on attention-output and MLP projections, response-only loss, two epochs, learning rate 10^{-4}, maximum sequence length 8,192, seed 0). Adapters were evaluated without retrieved guidance or teacher assistance.

Cost scope. The cloud-teacher ledger is capped at US$4.00 and recorded US$1.83 of cumulative use on 28 October 2026, including hosted decision-model calls booked at list price (about US$0.10) although they ran within the provider's free allocation. Earlier project reports quoted US$1.12 (under an earlier US$2.00 cap) and US$1.78; these are earlier snapshots of the same cumulative ledger, which is private and was not audited by a second party. The five consolidation training configurations total about 5.8 hours of wall time. GPU hours, student inference, hardware, and developer effort are excluded.

3. Benchmarks and analysis protocol

Kage-Stream contains four learned file families (subset selection, interval merge, log aggregation, CSV transformation), two families excluded from consolidation training (text normalization and configuration migration), and three read-only negative families. Its evaluation set has 24 in-family, 12 held-out-family, and 12 negative tasks. Kage-Web contains invoice, notification-settings, and hotel-selection fixtures (18 learned tasks), a held-out profile-form family, and read-only lookups; 30 tasks in all. Desktop-v1 contains fifteen tasks: six Calculator, six TextEdit, and three read-only System Settings tasks, split into ten training tasks and five variants.

Table 1. Evaluation units.

Experiment	Tasks	Repetition and interpretation
E-1 (files)	24 in-family, 12 held-out, 12 negative	two learning streams; raw baseline evaluated once and reused
C4 (browser)	30 (six per family)	three repeats per arm, same session within a round

Experiment	Tasks	Repetition and interpretation
G0 (desktop)	10 training + 5 variants	three repeats per task; variants reused during development
G0 confirmation	12 Calculator + 6 TextEdit, new	pre-registered; three repeats; all arms interleaved in one session
R3 series	24 learned-file, 12 held-out, 12 negative, 18 learned-web	three repeats per adapter; one training seed; R3b–R3e with single-run base anchors
D7 (deployment)	30 authored utterances per set	one prediction each; intent label only

Units. An *execution* is one attempt at one task; a *task* is one independent problem instance. Repeated executions are not independent samples, and a repeated execution is not a new task. Counts in tables are executions; tests are at the task level. "In-family" denotes new instances from the generators used for practice (different seeds and parameters), so in-family gains are same-generator generalization; the held-out families use different generators.

Task-level tests. Repeats are averaged within a task, and conditions are paired by task. We report the number of tasks that improved, declined, or tied, and the exact two-sided sign-flip probability of the mean within-task difference, computed by enumerating all 2^k sign assignments (by dynamic programming over integer-scaled differences, so 24- and 30-task sets are exact as well). A 95% task-bootstrap interval (10,000 resamples) is descriptive only. The minimum attainable value is $2/2^{k'}$ for k' non-tied tasks: six consistent tasks give 1/32, and three give 1/4. Values are nominal: they are not corrected for the number of contrasts, they assume exchangeable signs across tasks, and they condition on one trained adapter. No execution-level Fisher tests are reported. Absence of a detectable difference is not evidence of equivalence.

Sessions and development status. Within a C4 round and in the original R3-versus-base comparison, arms ran in the same session. R3b–R3e ran in later sessions, each with one base-model run as its anchor. Task manifests were frozen before the main runs, but failures on held-out families and on the desktop variants were inspected to design later prompts, tools, and variants. All scores except those in Sections 4.2 and 5.4 marked as confirmation are development-set scores. For the desktop suite we then collected a confirmation set: eighteen new tasks, written and committed with the analysis plan, the frozen rules, and the driver before any model ran on them (Section 5.4). No confirmation set exists for the file and browser suites.

4. Interface: observation and action representation

4.1 Browser settings: one observation field

The compact browser observation omitted the `checked` key for unchecked boxes. Settings tasks had resisted demonstrations, rules, and consolidation, and we had described them as a case where the student "knows but cannot do". In C4 round 1, the raw student ran the frozen web evaluation set under four interfaces with no learned guidance (Table 2).

Table 2. C4 browser interface ablation, round 1 (six settings tasks; all 30 tasks; three repeats).

Interface	Settings (/18)	All tasks (/90)	Change from V1
V1: unchecked state implicit	1	29	—
V2: explicit `checked: true/false`	14	41	one field
V2 + context packing	14	36	two changes
V2 + packing + state table	17	39	three changes

All six settings tasks improved from V1 to V2 (mean within-task difference +0.72, bootstrap 0.50–0.94, $p = 1/32$, the smallest value six tasks allow). Packing and the state table add no detectable task-level difference over V2 (two improved, one declined, three tied; $p = 1/2$), so 17/18 is a composite, not a single-field estimate. In round 2, the V2-plus-packing-plus-state-table configuration scored 18/18 on settings and 41/90 overall. A pre-save checker that compared decision-model goal extractions with the page made no intervention in 59 checks; with settings at ceiling, this benchmark gives no evidence for or against it.

Hotel failures persisted under every interface. Traces show the student selecting a hotel without opening its rating details, with or without context packing; we first blamed packing, and the corrected reading is procedural. This is a trace-grounded hypothesis, not a demonstrated fix.

4.2 Desktop TextEdit: action representation

With a whole-text `fill` action, TextEdit failures were mostly a missing trailing newline or a literal `\n` in the JSON string, although the append and write instructions state the newline requirement explicitly. Tools that let the program construct line breaks removed these failures (Table 3).

Table 3. TextEdit tool interfaces on six tasks (three training, three variants), three repeats each.

Tool interface	Fix (/6)	Append (/6)	Write (/6)	Primary (/18)	Second student (/18)
Whole-text `fill`	0	3	1	4	0
+ `replace_text`, `append_text` (v3)	6	2	0	8	—
Line-list actions (v4), no lessons	6	5	6	17	16
v4 + four lessons + plan (final)	5	4	6	15	14/17[a]

[a] One setup error (document window missing) excluded. The primary student's `fill` row combines training runs under interface v1 with variant runs under v2; both of the second student's `fill` runs are under v2.

For both students all six tasks improve from `fill` to v4 (primary: mean +0.72, $p = 1/32$; second student: mean +0.89, $p = 1/32$). For the primary student, v4 improves four tasks over v3 and ties two ($p = 1/8$). Adding the lessons and the plan at the same interface leaves four tasks unchanged and lowers two by one execution each (0/2/4, $p = 1/2$); the second student shows the same pattern (0/1/5, $p = 1$). The data therefore show no observed TextEdit increment from the lesson component once the interface is fixed, which is not the same as showing that the lessons are useless or harmful.

On six new TextEdit tasks of the pre-registered confirmation (Section 5.4), with no lessons and both interfaces interleaved in one session, `fill` scored 3/18 and v4 18/18 for the primary student, and 2/18 and 17/18 for the second student; all six tasks improved for each student (mean +0.83, $p = 1/32$, the minimum attainable with six tasks). The failures under `fill` were again missing trailing newlines.

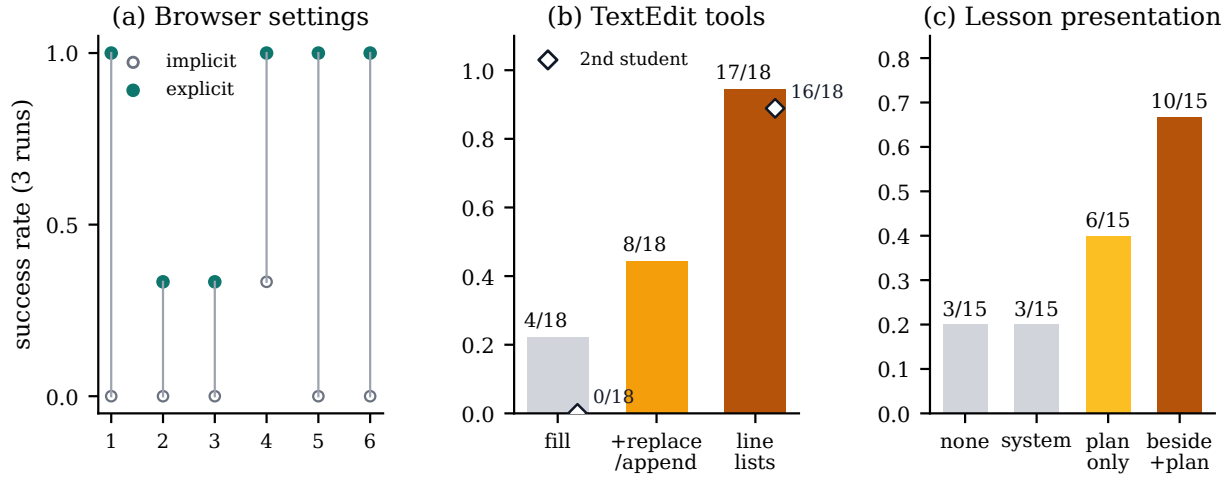


Figure 3: Interventions at three layers. (a) Browser settings per task under implicit (open) and explicit (filled) checkbox state, three repeats per task. (b) TextEdit tool interfaces; bars are the primary student, diamonds the second student; the primary `fill` bar mixes interface v1 training runs and v2 variant runs. (c) Desktop lesson-presentation pilot for the primary student on five variants, three repeats each; the second student scored 0–1/15 in every condition (Table 5).

4.3 The final desktop configuration

Line-oriented tools, reference-handling fixes, and task-adjacent lessons with planning together scored 41/45 executions for the primary student: 29/30 on the training tasks and 12/15 on the variants (Calculator 17/18, TextEdit 15/18, read-only System Settings 9/9). The earlier baselines were 19/30 on the training tasks under interface v1 without rules and 3/15 on the variants under interface v2 without rules. This is a configuration result. Table 4 lists the observed change on the training tasks by family together with the control that exists for it; it is not an attribution of the ten-point gain.

Table 4. Final configuration versus baseline on the ten training tasks (three repeats), by family.

Family	Baseline v1, no rules	Final	Matched control	Compatible explanation
TextEdit (3)	2/9	8/9	v4 without lessons: 8/9	line-oriented tools; no observed lesson increment
Settings (3)	7/9	9/9	none	the two baseline failures were a missing tool call and a whole-line reference, the cases the v2 fixes handle; no settings lessons existed, but the plan instruction applies
Calculator (4)	10/12	12/12	none	the gain is on the chain task whose failure produced the Calculator lesson (in-sample); lesson and plan not separated
Total	**19/30**	**29/30**		composite

Only the TextEdit change has a matched control. The settings change is consistent with the interface fixes by trace evidence but was not measured in isolation, and the Calculator change is an in-sample improvement that could come from the lesson, the plan, or run variation. A full fifteen-task v4 arm without lessons, interleaved with the final configuration in one session, is the most informative missing control.

The second student reached 18/44 executions under the same final configuration (Calculator 0/18, TextEdit 14/17, System Settings 4/9). Its Calculator failures were not repaired by any interface or prompt tested. Under the default references it pressed digit keys that did not match the operands (for 1234 + 5678 it began with 7, 8, 4); when references were replaced by control identifiers (interface v5) it pressed the digit keys in their listed order (Seven, Eight, Nine, Four, …). Calculator scored 0/18 under v5 as well. We first read the default-reference behavior as confusing reference numbers with digits; the v5 result does not support that reading.

5. Presentation-dependent adoption

5.1 Demonstrations help within a family

In E-1, after a 60-task learning stream, trajectory memory achieved 42/48 in-family executions and a skill library 39/48, compared with 14/48 for the raw baseline and 12/48 for budget-matched retries; the learning arms combine two streams evaluated on the same 24 tasks, and the raw baseline is one evaluation duplicated for the comparison. At the task level, memory improved 17 tasks and lowered one (mean +0.58, bootstrap 0.35–0.79, $p = 25/131{,}072$), and the skill library improved 14 and lowered none (+0.52, $p = 1/8{,}192$); memory and skills do not differ detectably (5/4/15, $p = 43/64$). On the twelve held-out tasks memory did not improve over the raw student (2/4/6, $p = 1/2$). Later single raw evaluations of the same 24 tasks ranged from 6 to 13 successes, which bounds how much of the difference a lucky baseline could explain but does not replace repeated baselines. Demonstrations taught same-generator procedures, not cross-family transfer.

5.2 Retrieval without enactment

On browser tasks, retrieved guidance was not consistently enacted. In C2 the relevant demonstration was present in the evaluation prompt, yet the student solved only a minority of invoice tasks, concentrated on the same two tasks. In C3, teacher-written procedural rules were injected through the system prompt (TF-IDF gated, at most

two rules), and invoice tasks rose from 0/24 to 14/24 executions: a missing-step failure repaired by a rule. The same delivery did not help settings tasks, whose failure was later traced to the observation field of Section 4.1, and hotel tasks remained procedural.

This appears to conflict with the desktop pilot below, where system-prompt rules did nothing. The two differ in more than delivery: the C3 rules supplied a missing step and were relevance-gated, whereas the desktop rules concerned action representation (line breaks) and Calculator operator order. We do not isolate content type, retrieval, number of rules, or planning.

5.3 The desktop lesson pilot

Calculator chain tasks exposed a missing procedure: entering an addition followed by a multiplication without first resolving the addition gives the wrong grouping. The teacher wrote one rule about pressing Equals after the addition and three about preserving text and newlines. On the five variants, which were not used to write the rules, four conditions were compared under interface v2 (Table 5).

Table 5. Desktop lesson pilot: five variants, three repeats each (executions).

Prompt condition	Primary: Calc. (/6)	Primary: TextEdit (/9)	Primary total (/15)	Second student (/15)
No lessons	1	2	3	0
Lessons in system prompt	1	2	3	1
Plan first, no lessons	4	2	6	0
Lessons beside task; explain application, then act	5	5	10	0

For the primary student the combined condition is numerically strongest, and planning alone accounts for much of the Calculator change. Against no lessons, the combined condition improves three tasks and ties two (mean +0.47, bootstrap 0.13–0.80, $p = 1/4$); plan only improves two and ties three ($p = 1/2$); lessons in the system prompt are exactly neutral on average ($p = 1$); and the combined condition improves two tasks over plan only and ties three ($p = 1/2$). The combined condition changes lesson placement and adds an application step, so it does not isolate rule content, location, or deliberation. The second student's lessons were written from its own failures and were generic ("clear the display, verify the expression"), and every condition scored 0 or 1 of 15. The pilot therefore shows that the same rules can be ignored in one presentation and used in another by the primary student; it does not establish a general effect.

5.4 Pre-registered confirmation

The pilot confounded placement with planning and reused variants that the developer had inspected. We therefore wrote twelve new Calculator tasks and six new TextEdit tasks, verified them with scripted solutions (18/18), and committed the tasks, the four frozen rules from Section 5.3, the driver, and an analysis plan with four primary contrasts before the first model run. Ten Calculator tasks require grouping, as in $(61 - 25) \times 3$; two *precedence controls* ($8 + 7 \times 6$, $12 + 9 \times 3$) are solved by typing straight through, so pressing Equals after the addition, as the rule advises, gives the wrong answer. Five conditions ran under interface v4: no rule, rule in the system prompt, plan only, rule beside the task without a plan request (new), and rule beside the task with the application plan. For every task and repeat, all conditions ran back to back in a rotated order, so time and application state are spread across conditions. Two executions failed in setup three times (Calculator had no window) and count as failures, as pre-registered; excluding them changes only the second contrast, to 7/0/3 and $p = 1/64$.

Table 6. Pre-registered desktop confirmation, primary student, three repeats per task (executions).

Condition	Grouping (/30)	Precedence controls (/6)
No rule	1	6
Rule in system prompt	6	6

Condition	Grouping (/30)	Precedence controls (/6)
Plan only	12	6
Rule beside task, no plan	19	4
Rule beside task, plan	23	4

The four pre-registered contrasts on the ten grouping tasks (improved/declined/tied, exact p) are: beside with plan versus no rule, 10/0/0, $p = 1/512$; beside without plan versus system prompt, 8/0/2, $p = 1/128$; beside with plan versus plan only, 6/1/3, $p = 1/16$; and beside with plan versus beside without plan, 5/1/4, $p = 7/32$. The first two meet the pre-registered criterion ($p < 0.05$ in the predicted direction); the last two do not. The combined condition of the pilot therefore replicates on new tasks, but most of its effect comes from where the rule appears: placing it beside the task, without asking for a plan, raised success from 6/30 to 19/30, while the plan's further increment was not detectable. On the precedence controls, the three conditions without a task-adjacent rule scored 18/18 and the two with one scored 8/12; three of the four failures pressed Equals after the addition, producing $(12 + 9) \times 3$ or $(8 + 7) \times 6$. The second student was not run on these Calculator tasks, because it had scored 0 in every earlier Calculator condition (0/108 executions), and the plan said so before the run.

These results narrow C2: for this student and this rule, placement beside the task governs adoption, and it governs it indiscriminately. They do not separate position in the context from framing as part of the request, and they rest on one rule, one family, one student, and one session.

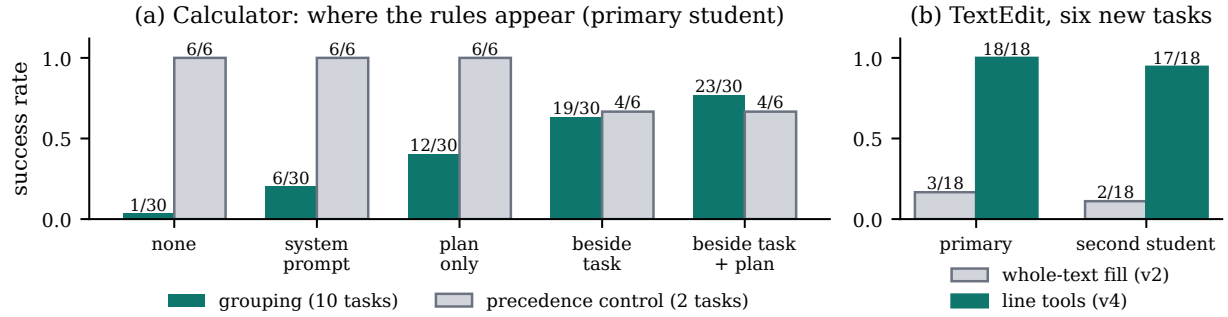


Figure 4: Pre-registered confirmation on new tasks. (a) Calculator, primary student: ten grouping tasks and two precedence controls, three repeats each, under five presentations of the same rule. (b) TextEdit, six tasks, three repeats, for both students.

6. Consolidation and retention

6.1 Data and training units

The R2 archive contains 1,900 action-level examples from 151 unique task IDs. Each row pairs a conversation context with a target response, and one successful rollout can contribute several rows. The four collection reports record 704 attempts and 472 successes before filtering, and the retained archive contains 446 distinct task/source/attempt combinations; none of these counts is a number of independent tasks. R3 splits the archive by task into 1,677 training and 223 validation examples. Replay data come from AgentNet desktop-action steps (Wang et al., 2025; 800 examples) and Toucan MCP tool calls (Xu et al., 2025; 600 examples); neither contains code-writing steps. Table 7 lists the training exposure of each variant.

Table 7. Training exposure (saved training configurations; seed 0, two epochs).

Version	Main change	Training examples	Replay	Optimizer steps
R3	guided-success targets	1,677	0	420
R3b	filter reasoning-free targets; random replay	2,781	1,400	696

Version	Main change	Training examples	Replay	Optimizer steps
R3c	filter; LESS-selected replay	2,266	879	567
R3d	restore targets with teacher rationales; LESS replay	2,736	879	684
R3e	rationales; random replay	3,251	1,400	813

R3e's configuration lists 3,257 rows, six of which exceed the 8,192-token limit, leaving 3,251 trained examples.

6.2 Acquisition and loss

Table 8. Consolidation results (executions; three inference runs per adapter). Anchors are untrained base runs from the same session: three runs for R3, one run for R3b–R3e.

Version	Anchor in / held-out	Learned files (/72)	Held-out files (/36)	Learned web (/54)	Negative skill misuse (/36)
Base (R3 anchor)	30/72 · 15/36	30	15	10	0
R3	same as base	64	1	21	2
R3b	10/24 · 7/12	34	15	31	0
R3c	11/24 · 7/12	34	2	22	0
R3d	6/24 · 5/12	62	6	20	0
R3e	11/24 · 7/12	65	1	22	0

In the same-session R3 comparison, the in-family gain appears in every repeat (22, 22, 20 of 24 versus 9, 12, 9) and so does the held-out decline (1, 0, 0 of 12 versus 5, 5, 5). At the task level, 14 in-family tasks improved, 2 declined, and 8 tied (mean +0.47, bootstrap 0.28–0.67, $p = 11/32{,}768$); among held-out tasks 6 declined, 1 improved, and 5 tied (−0.39, bootstrap −0.67 to −0.11, $p = 3/64$). Learned web tasks improved on 7 and declined on 2 (+0.20, $p = 7/64$). Against their single-run anchors, R3d and R3e show the same pattern (in-family $p = 1/32{,}768$ and $3/4{,}096$; held-out −0.25, $p = 33/128$, and −0.56, $p = 1/64$), whereas R3b and R3c show no detectable in-family gain (+0.06 and +0.01) with held-out differences of −0.17 ($p = 131/256$) and −0.53 ($p = 1/64$). Single-run anchors are noisier than the three-run base, so these variant contrasts are descriptive.

R3 also issued skill calls on read-only negative tasks in 2 of 36 executions; both were on the same look-alike task (two of its three runs) and failed it, and neither run wrote a file. The base model and R3b–R3e issued none. Under the round-2 browser interface, R3e achieved 7/18 invoice executions versus 0/18 for the same-interface raw arm and 47/90 versus 41/90 overall; at the task level the overall gain comes from three tasks, all invoices (three improved, one declined, 26 tied; $p = 1/4$). Some invoice behavior survives the interface change; broad interface-invariant transfer is not shown. A 30-question general-ability check scored 27–28 across variants, too small to establish preservation of general capability.

6.3 What the variants do not identify

Filtering reasoning-free targets disproportionately removed file solutions: the learned families contain 202 reasoning-free `write_file` targets and 101 with reasoning (102 including one negative-family row), so the filter did not remove every file-writing example. Restoring the filtered targets with teacher-written rationales recovered most of the in-family gain while held-out success stayed low (R3d, R3e). This rules out the account that restoring visible reasoning text is sufficient to prevent the degradation in these configurations.

It does not show that gain and loss share a single cause. R3b and R3c filter the same targets and reach the same 34/72 in-family, yet their held-out results differ (15/36 versus 2/36). They also differ in replay algorithm, replay size (1,400 versus 879), source mixture, and optimizer steps (696 versus 567), so the contrast shows that retention is

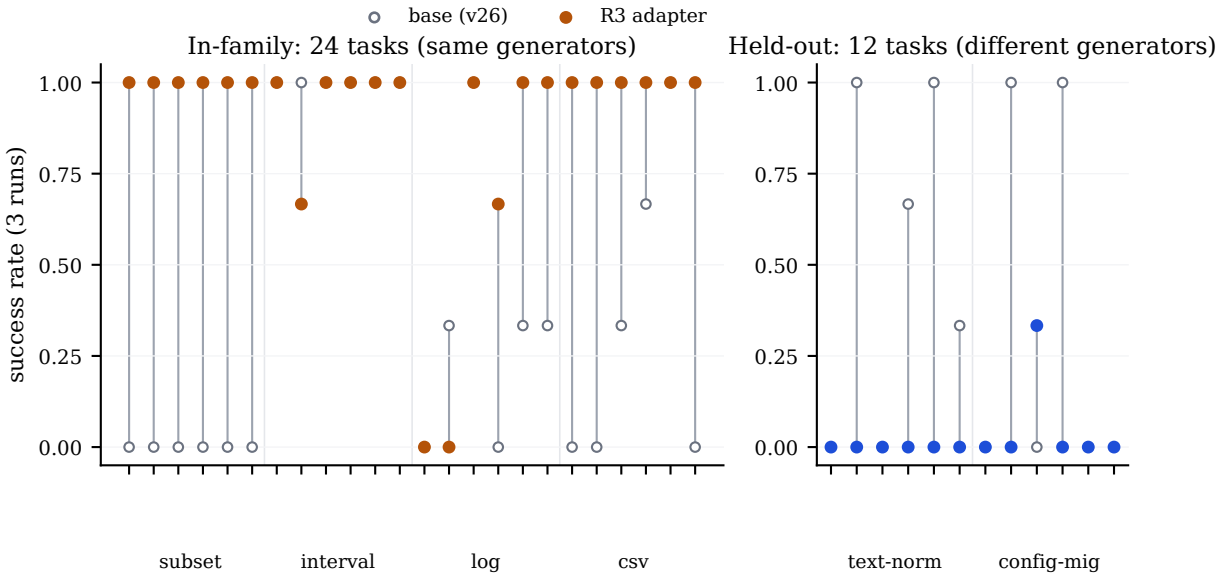


Figure 5: Per-task success of the base model (open) and the R3 adapter (filled), three runs each. Left: 24 in-family tasks, of which 14 improved, 2 declined, and 8 tied. Right: 12 held-out-family tasks, of which 6 declined, 1 improved, and 5 tied.

not a function of filtering alone, not that replay selection is responsible. R3b's held-out 15/36 equals the three-run base from the R3 session and lies below its own single-run anchor (7/12, i.e. 21/36 at that rate); the difference is not detectable at the task level ($p = 131/256$), and R3b also kept little of the in-family gain.

Trace analyses report less reading back of written files and plausible but incorrect code after training. Reduced verification is a candidate mechanism, not an established cause: no intervention forced read-back while holding model and task fixed, and R3b retained more held-out success despite reduced verification. Testing it requires controlled verification interventions and training data that keep successful checking steps; testing replay explanations requires matched amounts, source composition, exposure, and several training seeds. None of these experiments is claimed here.

7. Second student

The desktop replication with Nemotron-3-Nano-4B ran the G0 protocol unattended in one session, with lessons written by the same teacher from the second student's own training failures (US$0.0014). Two earlier attempts were void and are excluded: one failed in Calculator setup when the display was not clear, and one ran after the screen locked, which disables Accessibility actions. Its training-task baseline was 3/30 under interface v2 (primary student: 19/30 under v1). Three observations follow. The TextEdit tool effect reproduced (Table 3) and held on the new confirmation tasks (2/18 to 17/18). The presentation effect did not (Table 5). The Calculator failure was specific to this student and survived two interface changes (Section 4.3). The replication covers one model and one session and was not reviewed by a second person; it narrows C1 to a tool-representation effect seen in two students and C2 to the primary student.

8. Discussion

Why interfaces matter more for small students. The two interface failures share a structure: the information the student needed was present but had to be inferred or encoded. In the V1 browser observation an unchecked box was signalled by the *absence* of a key, while the same entry still carried `"value": "on"` (Appendix B.1); a larger model may infer the convention, but a 4B student reads what is written. In whole-text editing, a line break had to be emitted as an escape sequence inside a JSON string, and the student produced either nothing or a literal backslash-n even when its own plan stated the requirement. In both cases the repair removed an inference or an encoding step instead of teaching one. This suggests a cheap diagnostic before collecting data or training: hold the guidance fixed and change only the representation of the observation or action. In our runs, a failure that moved

under such a change was not a capability gap, and the repair was a one-time engineering change rather than a recurring data cost.

Placement decides whether a rule is used, and where. For the primary student, the same rule was mostly ignored in the system prompt and used when it sat beside the task; asking for an application plan added little once the rule was there. The same placement made the student apply the rule to expressions where it did not hold. A rule beside the task thus acts less like advice the student weighs and more like an instruction it follows, so deployed rules need stated conditions of use and should be tested on cases where they do not apply. **Guidance needs a competence floor.** The second student did not benefit from any presentation: it could not map an arithmetic expression onto Calculator keys, and no rule about operator order could repair that. Presentation effects are therefore conditional on the student already being able to carry out the underlying actions; where that floor is missing, the failure belongs to an earlier layer (here, grounding operands to controls) that rules do not reach.

Measuring retention is cheap; skipping it is not. The in-family gains came from new instances of the same generators, which is a narrow form of generalization, and the two excluded families exposed losses that in-family scores alone would have hidden. Evaluating excluded families and read-only negatives after every weight change cost little relative to training. In deployment we therefore kept learning in context (memory, skills, and rules behind a relevance gate) rather than shipping a consolidated adapter; family-gated adapters are an untested alternative.

Privileged teacher feedback can reach the student. The desktop teacher saw expected values and checker findings, and one of the rules it wrote for the student cites the checker directly ("since the checker requires each line to be newline-terminated", Appendix B.3). The rule is general and contains no test answer, but it shows that feedback given to a teacher can shape what the student is told. Evaluations that separate student-visible from teacher-visible information should inspect the generated guidance, not only the prompts.

9. Related work

Computer-use benchmarks. OSWorld evaluates multimodal agents on open-ended tasks in real desktop environments (Xie et al., 2024), and WebArena on self-hosted websites (Zhou et al., 2024); both report large gaps between the strongest agents and humans. Our suites are smaller and procedurally generated so that families can be held out of training and failures inspected one interface field at a time; they are not a substitute for these benchmarks.

Interfaces. SWE-agent studies agent-computer interface design and shows that tools adapted to language-model agents improve software-engineering performance (Yang et al., 2024). Our narrower contribution is a diagnostic study in which explicit checkbox state and line-oriented actions address observed small-model failure modes; we do not claim interface design as a new principle. GUI toggle state has independently been identified as a failure point for multimodal agents (Wu et al., 2026). The ARTEMIS Android automation project prompted our interface ablation.

Experience and consolidation. Agent Workflow Memory induces and retrieves reusable procedures (Wang et al., 2025). Context distillation transfers information supplied in context into model behavior (Snell et al., 2022). Kage's guided re-execution draws on the recursive self-rewrite component of Terminal-RSI (Yang et al., 2026), but not its reinforcement-learning stages or a second training round. Our rationale step is related to STaR (Zelikman et al., 2022), except that a teacher writes rationales for existing targets. Harness search changes the scaffolding around a model rather than its weights (Meta-Harness, Lee et al., 2026; Harness^2, Xu et al., 2026); the interface changes reported here were made by the developer.

Retention and data selection. Biderman et al. (2024) compare LoRA with full fine-tuning and find that LoRA learns less and forgets less; this is a relative comparison, it does not imply that LoRA avoids forgetting, and we have no full-fine-tuning comparator. LESS selects examples aligned with a target capability (Xia et al., 2024); our unequal-size replay comparison cannot establish a general limitation of LESS as an anti-forgetting method. Forgetting from synthetic fine-tuning data has been reported for the retrieval component of LLM-agent skill systems (Murtaza et al., 2026), and cross-environment transfer of fine-tuned agents has been studied in relation to observation and action shifts (Xi et al., 2026).

Position of information in context. Language models use information less reliably when it sits in the middle of long contexts than at the beginning or end (Liu et al., 2024). Moving rules from the system prompt to beside the task changes their position as well as their framing; our confirmation separates placement from planning but not position from framing.

Statistical practice. Our task-level analysis follows the argument that significance tests must respect the unit of independent sampling (Dror et al., 2018).

10. Limitations and reproduction status

The study uses one primary model, a partial desktop replication with a second, one training seed per adapter, procedural task families, a small desktop suite, and one user's deployment. Repeated runs characterize inference variability but do not add independent tasks or training replicates. Developer inspection of evaluation failures influenced later prompts, tools, and training variants. A pre-registered confirmation exists only for the desktop tool and rule-placement results; it covers one rule family, eighteen developer-written tasks, and one session per student. Cross-session single-run anchors for R3b–R3e, unequal training exposure across variants, a composite final desktop configuration, and nominal tests without multiplicity correction further limit causal interpretation.

Deployment note. In daily use, a hosted decision model (Clef) routed requests to program handlers and matched the intent label on 30/30 utterances in each of two authored sets, against 15/30 and 16/30 for the pattern router it replaced; with a 0.8 confidence fallback to the local model, 30/30 and 28/30. These are intent-label matches only, not completed tasks: one second-set date label is itself wrong, the first set was reused as development data for the second router, there is no matched 4B router baseline, and request text leaves the machine for routing. Before-and-after cases from the private activity log (a weather question from 36.5 s to 0.8 s; a knowledge question from a failed 16 s tool loop to a correct 2.3 s answer) are illustrations, not latency distributions.

Reproduction status. The repository contains source code, frozen manifests, raw evaluation traces for the principal runs (including the second-student runs in `evidence/reports/g0-nemotron`), training configurations, and offline scripts that recompute every table and test from the raw archives (`docs/paper/v0.5-support/verify.py`, `make_figures.py`). Reproduction is incomplete: adapters are in private storage, the evidence checksum index stops at archive v31 (v32, v33, G0, D7, and the second-student runs are not covered), and the daily-use logs contain private content. A release should provide adapters or reconstruction instructions, complete checksums, environment versions, and sanitized timing data.

Use of AI assistance. Code, data analyses, figures, and consistency checks were developed with AI coding assistants. Experimental design decisions, the reported numbers, and the claims are the author's responsibility and were checked against the released evidence.

11. Conclusion

In this case study, small-agent failures responded differently to interface repair, the presentation of guidance, and consolidation. Explicit observation fields and line-oriented action tools produced large development-set improvements, and the tool effect held for a second 4B student. Both effects were confirmed on new, pre-registered tasks. Rules placed beside the task were used, and the plan added little; the same placement made the student apply a rule where it did not hold, and the effect did not reproduce with the second student, which lacked the underlying Calculator skill. Consolidation improved trained families while degrading two excluded families; follow-up variants narrow the explanations without identifying a mechanism, and the adapter once misused a skill on a read-only task. The practical lesson is to examine the interface and the presentation of guidance before attributing a failure to capability, and to measure retention whenever weights change.

References

- Biderman, D., Portes, J., Gonzalez Ortiz, J. J., Paul, M., Greengard, P., Jennings, C., King, D., Havens, S., Chiley, V., Frankle, J., Blakeney, C., & Cunningham, J. P. (2024). *LoRA Learns Less and Forgets Less.* Transactions on Machine Learning Research. arXiv:2405.09673.
- Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). *The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing.* ACL 2018, 1383–1392. doi:10.18653/v1/P18-1128.
- Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., & Finn, C. (2026). *Meta-Harness: End-to-End Optimization of Model Harnesses.* arXiv:2603.28052.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). *Lost in the Middle: How Language Models Use Long Contexts.* Transactions of the Association for Computational Linguistics. arXiv:2307.03172.
- Murtaza, S. S., Nie, Y., Soni, U., Wen, E., & Frydenlund, A. (2026). *When Synthetic Data Hurts: On Catastrophic Forgetting in Skill Retrieval for LLM Agents.* EMNLP 2026 Industry Track. arXiv:2609.10750.
- Snell, C., Klein, D., & Zhong, R. (2022). *Learning by Distilling Context.* arXiv:2209.15189.

- Wang, X., Wang, B., Lu, D., Yang, J., Xie, T., et al. (2025). *OpenCUA: Open Foundations for Computer-Use Agents* (AgentNet). NeurIPS 2025. arXiv:2508.09123.
- Wang, Z. Z., Mao, J., Fried, D., & Neubig, G. (2025). *Agent Workflow Memory*. ICML 2025. arXiv:2409.07429.
- Wu, Z., Mao, R., Tian, Z., Cheng, P., Ju, T., Wu, Z., Dong, L., Sheng, H., Zhang, Z., & Liu, G. (2026). *See, Think, Act: Teaching Multimodal Agents to Effectively Interact with GUI by Identifying Toggles*. CVPR 2026. arXiv:2509.13615.
- Xi, Z., Guo, X., Liu, J., Zhang, J., Fan, Y., Zhang, Z., Liu, S., Chai, M., Shi, X., Zhai, Y., Cai, X., Gui, T., Zhang, Q., & Huang, X. (2026). *Can RL Improve Generalization of LLM Agents? An Empirical Study*. arXiv:2603.12011.
- Xia, M., Malladi, S., Gururangan, S., Arora, S., & Chen, D. (2024). *LESS: Selecting Influential Data for Targeted Instruction Tuning*. ICML 2024. arXiv:2402.04333.
- Xie, T., et al. (2024). *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. NeurIPS 2024. arXiv:2404.07972.
- Xu, R., Chen, Y., CuiZhu, Z., Dalvi Mishra, B., Ming, Y., Yu, H., Han, R., Lee, C.-Y., & Pfister, T. (2026). *Harness²: Recursive Agent Harnessing for an Open World*. arXiv:2610.14224.
- Xu, Z., Meza Soria, A., Tan, S., Roy, A., Agrawal, A. S., Poovendran, R., & Panda, R. (2025). *TOUCAN: Synthesizing 1.5M Tool-Agentic Data from Real-World MCP Environments*. arXiv:2510.01179.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. arXiv:2405.15793.
- Yang, J., Li, Z., Shi, Y., Li, Z., Wang, R., Zhou, R., Mi, H., Yan, S., & Liang, L. (2026). *Terminal-RSI: Recursive Self-Improving Terminal Agents via Recursive Self-Rewrite and Policy-Refine*. arXiv:2610.22192.
- Zhou, S., et al. (2024). *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR 2024. arXiv:2307.13854.
- Zelikman, E., Wu, Y., Mu, J., & Goodman, N. D. (2022). *STaR: Bootstrapping Reasoning With Reasoning*. NeurIPS 2022. arXiv:2203.14465.

Appendix A. Evidence map

Paths are relative to the repository root. Historical reports keep their original interpretations; this paper states the revised ones.

Result	Evidence
File learning (E-1)	`evidence/kaggle/v6`, `v7`; `evidence/reports/e1-report`
Browser guidance (C2, C3)	`evidence/reports/c2`, `c3`
Browser interfaces (C4)	`evidence/kaggle/v28` (round 1), `v33` (round 2); `evidence/reports/c4`
Desktop tools and lessons (G0)	`evidence/reports/g0/*` (configs, results, traces, teacher log)
Second student	`evidence/reports/g0-nemotron/*`
Desktop confirmation	`docs/plans/g0-confirm-prereg.md` (pre-registration); `eval/desktop_v2_confirm/tasks.json`; `evidence/reports/g0-confirm/` (`raw/`, `raw-nemotron/`, `analysis.json`)
Guided training data (R2)	`evidence/reports/r2/sft-v21.jsonl.gz` and four collection statistics files
Consolidation (R3 series)	`evidence/kaggle/v26`, `v29`–`v32`; `evidence/reports/r3*`; saved `train-config.json` files
Deployment routing (D7)	`evidence/reports/d7`; `eval/intent_v1`
Tables, tests, figures	`docs/paper/v0.5-support/verify.py` (writes `verify.json`); `docs/paper/v0.7-support/make_figures.py`

Appendix B. Examples

B.1 Browser observation, V1 and V2

The same unchecked box in the compact observation of a settings task (C4 round 1, task `web_prefs-11000`, first step; form fields omitted). Under V1 the unchecked state is signalled only by the missing `checked` key, while `value`

still reads `"on"`:

```
V1: {"target_ref":"e5","label":"Security alerts","tag":"input","operations":["click"],
     "value":"on","input_type":"checkbox","name":"security"}
V2: {"target_ref":"e5","label":"Security alerts","tag":"input","operations":["click"],
     "value":"on","checked":false,"input_type":"checkbox","name":"security"}
```

B.2 TextEdit actions

Whole-text interface: `fill(ref, text)` replaces the whole text. Line-oriented interface (v4): `replace_text(ref, old, new)` replaces one exact occurrence; `append_lines(ref, lines)` and `set_lines(ref, lines)` take a list of strings, and the program writes a line break after each line. Two failures under `fill`, as saved on disk (primary student, interface v2):

```
te_append2 (instruction: add "- call Ann" ... ending with a newline)
  saved: "To do:\n- email Bob\n- call Ann"          (final newline missing)
te_fix2
  saved: "Please call me tomorrow.\\nThanks.\\n"     (literal backslash-n: one line)
```

B.3 Lesson presentation

The four rules written by the teacher from failures on the training tasks (verbatim):

```
TextEdit 1: Before editing a document's text, read its full current contents and preserve
  every character, including trailing whitespace and line breaks; never retype the content
  from memory.
TextEdit 2: Always ensure the document ends with a trailing newline when the required format
  specifies one, and verify the saved file content matches the expected format exactly
  before finishing.
TextEdit 3: Always ensure the document ends with a trailing newline after the final line
  before saving, since the checker requires each line to be newline-terminated.
Calculator: When an expression mixes addition and multiplication, press Equals after the
  addition to resolve that part before entering the multiplication, or otherwise ensure
  the grouping you intend is actually applied.
```

System-prompt condition: the rules are appended to the system prompt under "Lessons learned from earlier tasks in this app (follow them):". *Beside-the-task condition*: they follow the task instruction in the user message under "Lessons from your earlier attempts at this kind of task:", followed by "Before your first action, write a short plan that says how each lesson applies to this task (for example the exact text or the exact button order you will use). Then follow the plan." *Plan-only control*: no rules, and "Before your first action, write a short plan for this task (for example the exact text or the exact button order you will use). Then follow the plan."

Appendix C. Reproduction

Result	How to recompute or rerun
All tables and task-level tests	`python3 docs/paper/v0.5-support/verify.py` (reads `evidence/`; writes `verify.json`)
Confirmation analysis	`python3 evidence/reports/g0-confirm/analyze.py` (writes `analysis.json`)
Figures	`python3 docs/paper/v0.7-support/make_figures.py`
Desktop runs (G0, second student)	`scripts/experiments/desktop_run.py --arm a0 --runs 3 --tasks … --interface v2/v4/v5 [--lessons … --lesson-mode plan/plan-only]`; `scripts/experiments/g0_replicate.sh` with `KAGE_MODEL_PATH` for another student; confirmation `scripts/experiments/g0_confirm.py --out … [--experiments x1,x2]`

Result	How to recompute or rerun
File and browser runs (E-1, C4, R3 evaluation)	`scripts/experiments/kaggle_e1.py --phase e1 / c4 / c4b / r3-eval / adapter-eval` on a Kaggle T4 (`docs/development/kaggle-runbook.md`)
Consolidation training	phases `r3-train` … `r3e-train` of the same script on an A100; configurations in `evidence/reports/r3*/train-config.json`
Paper	`docs/paper/v0.7/build.sh` (pandoc + tectonic)

Not reproducible from the repository alone: the trained adapters (private storage), the daily-use activity logs (private content), and the teacher ledger.